SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	M. SANA FATHIMA	**Reg. No.:**	192371082
Date:	29/08/2025	**Duration:**	60 minutes

Code of Conduct

I M. SANA FATHIMA (192371082) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. SANA FATHIMA_____

Annexure A – CI/CD Pipeline Design Exercise

Assigned Problem 9 – E COMMERCE ORDER FULFILLMENT PLATFORM

Introduction

The E-Commerce Order Fulfillment Platform manages the complete lifecycle of an online order, from customer registration and product selection through cart management, checkout, payment, inventory, warehouse processing, packing, shipment, delivery, and tracking. Because many connected components are involved, code changes must be integrated, tested, quality-checked, secured, packaged, and deployed in a controlled manner. A CI/CD pipeline automates these activities and provides repeatable delivery. The assessment requires analysis of CI/CD needs, a workflow covering source, build, testing, quality analysis, packaging and deployment, suitable tools with justification, automated testing, development/testing/production environments, security, notifications, rollback, and a pipeline diagram.

CI/CD Requirements

The platform requires frequent and reliable integration because changes may affect authentication, products, carts, payments, inventory, warehouse processing, shipment, and tracking. Every change should pass source-control review, automated build and test stages, code-quality checks, security checks, packaging, and staging validation before production release. The pipeline should retain logs and artifacts, notify the team about failures and successes, protect production credentials, and provide rollback to the last stable release.

Pipeline Architecture and Workflow

The proposed architecture uses GitHub for source-code management and GitHub Actions for automation. Developers work on feature branches and submit pull requests. The pipeline checks out the code, installs dependencies, builds the application, runs unit tests, performs SonarQube quality analysis, executes security checks, packages the application into a Docker image, and stores the image in a container registry. The image is deployed to Kubernetes staging, where API, integration, and smoke tests are executed. A successful staging result moves to an approval gate. After approval, the same tested image is deployed to Kubernetes production. Health checks and monitoring verify the release, and a failed deployment can be rolled back.

Source Code Management

GitHub provides version control, branch management, pull requests, review, and traceability. A protected main branch should prevent direct unreviewed changes. Feature branches should be used for individual changes. Pull requests should require review and successful automated checks before merging. Each release should retain its source commit identifier so that a production deployment can be traced to the exact code revision.

Build Stage

The build stage installs dependencies, compiles or bundles the application, and produces a deployable artifact. Maven can be used for a Java backend and npm for JavaScript-based components. Build failures should immediately stop the pipeline. Build logs should be retained to support troubleshooting.

Automated Testing Strategy

Unit tests should run on every pull request because they provide fast feedback. Integration tests should verify communication between order, inventory, payment, and fulfillment components. API tests should cover authentication, product, cart, order, payment, tracking, and cancellation endpoints. Smoke tests should run after staging deployment to confirm critical functions. Regression tests should run before production release. Test coverage thresholds can be configured as a quality gate.

Code Quality Analysis

SonarQube is proposed for code-quality analysis. It can identify bugs, code smells, duplication, maintainability issues, and coverage-related problems. A quality gate should prevent promotion when configured quality conditions are not met.

Security Integration

Security should be part of the pipeline. Static analysis can identify insecure coding patterns, dependency scanning can identify vulnerable third-party packages, secret scanning can detect accidentally committed credentials, and container scanning can identify vulnerable images. Production credentials should be stored in protected secret storage and accessed only by authorized workflows.

Packaging and Artifacts

Docker is proposed for packaging the application as a consistent container image. Each image should have an immutable version or commit-based tag and be stored in a secure registry. Only images that pass testing, quality, and security gates should be eligible for deployment. The exact image tested in staging should be the one promoted to production.

Deployment Environments

Three environments should be used: development, staging, and production. Development supports active coding and early checks. Staging should closely resemble production and should run integration, API, smoke, and release-candidate tests. Production is the live customer environment and should have the strictest access and approval controls.

Rollback Strategy

Every production release should retain the previous stable version. If health checks fail, error rates increase, or a critical business defect appears, the deployment should be rolled back to the previous stable image. Kubernetes deployment history can support controlled rollback. Database changes must be designed carefully so rollback does not corrupt orders, payments, inventory, or fulfillment records.

Notifications and Monitoring

The pipeline should notify the development and operations teams when builds, tests, quality gates, security checks, or deployments succeed or fail. Logs should be retained. Production monitoring should observe application health, response times, error rates, resource use, and critical orderprocessing services. Alerts should be configured for serious failures.

Example End-to-End Workflow

A developer pushes a feature branch and opens a pull request. GitHub Actions checks out the code, installs dependencies, builds the application, and runs unit tests. Successful tests trigger quality and security analysis. The application is packaged into a Docker image and pushed to the registry. The image is deployed to Kubernetes staging, followed by API, integration, and smoke tests. Failure stops the pipeline and sends a notification. Success moves to the approval gate. After approval, the same tested image is deployed to production. Health checks and monitoring verify the release, while rollback restores the previous stable version if necessary.

Tool and Technology Justification

GitHub is appropriate for source management and collaboration. GitHub Actions automates the workflow. Maven or npm manages builds and dependencies according to implementation technology. JUnit or Jest provides automated unit testing. SonarQube provides code-quality analysis and quality gates. Docker provides consistent packaging. Kubernetes supports deployment, scaling, health checks, and rollback. A container registry provides versioned artifact storage. Security scanning tools can be integrated into the pipeline for source, dependency, secret, and image checks.

Security, Reliability and Ethical Considerations

Production access should be restricted and deployment credentials should be protected. Secrets must not be committed to source control. Every deployment should be traceable to an approved source revision. Automated gates should not be bypassed merely to accelerate release. Reliability is especially important because the platform handles orders and payment-related information. The pipeline should stop unsafe releases, retain traceable artifacts, monitor production, notify responsible teams, and support rollback. The assessment emphasizes robust testing, quality assurance, DevOps practices, security, reliability, and ethics.

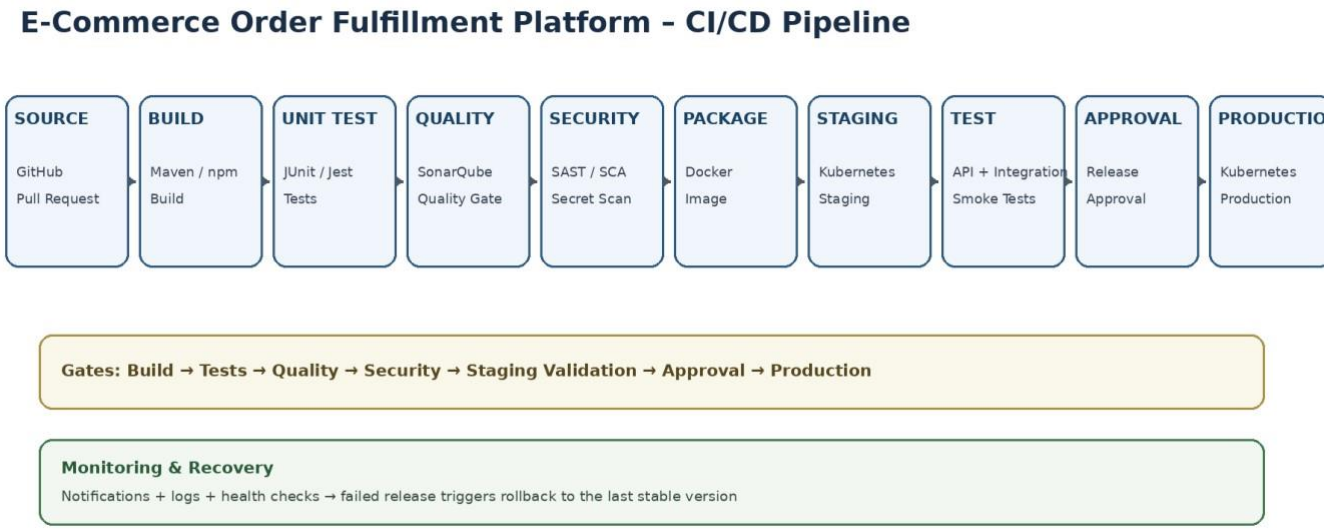
Expected Outcomes

The expected outcome is a repeatable and automated delivery process in which every meaningful change is validated before deployment. The pipeline should reduce manual errors, identify defects early, improve code quality, detect security issues before release, maintain consistent artifacts, and provide controlled movement from development to staging and production. Failed builds or tests should stop promotion, security problems should block unsafe releases, and failed production deployments should be recoverable through rollback.

Deliverables

The CI/CD Pipeline Design deliverable should contain the CI/CD requirements analysis, architecture and workflow, pipeline stages, tool and technology selection with justification, automated testing strategy, deployment environments, security practices, quality gates, notifications, monitoring, rollback strategy, pipeline diagram, workflow explanation, and justification based on the assigned problem statement. These correspond to the exercise deliverable requirements.

CI/CD Pipeline Diagram



CI/CD Figures

The following figures provide additional visual explanations of the proposed CI/CD workflow, automated testing, deployment environments, security gates, rollback strategy, and monitoring process.

Figure 2 – Git Branching and Pull Request Workflow

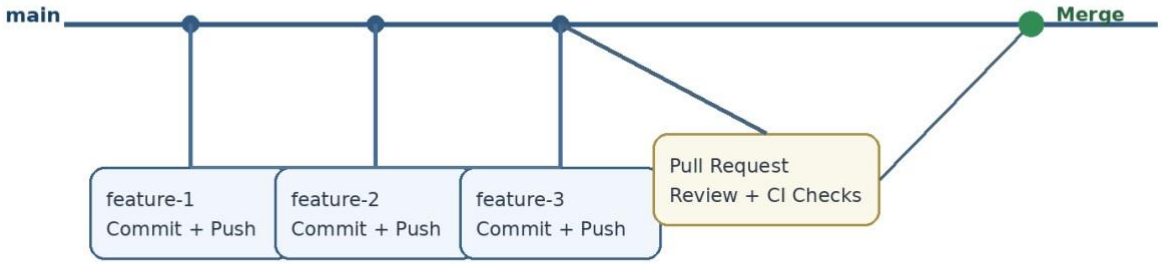
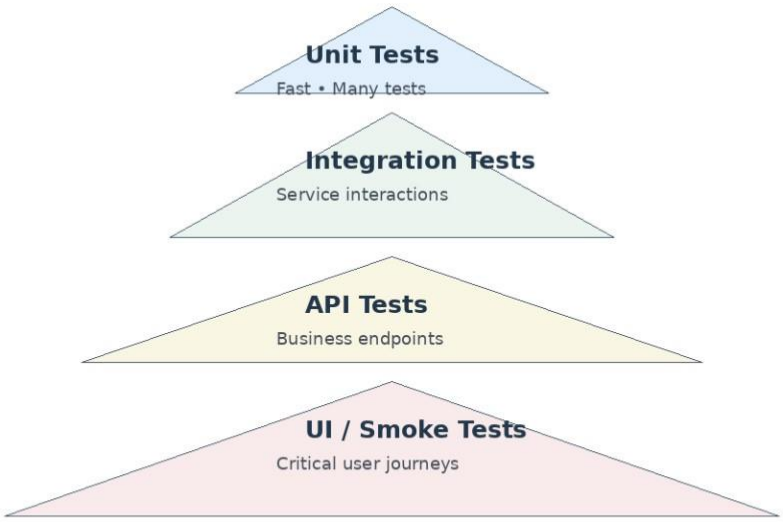
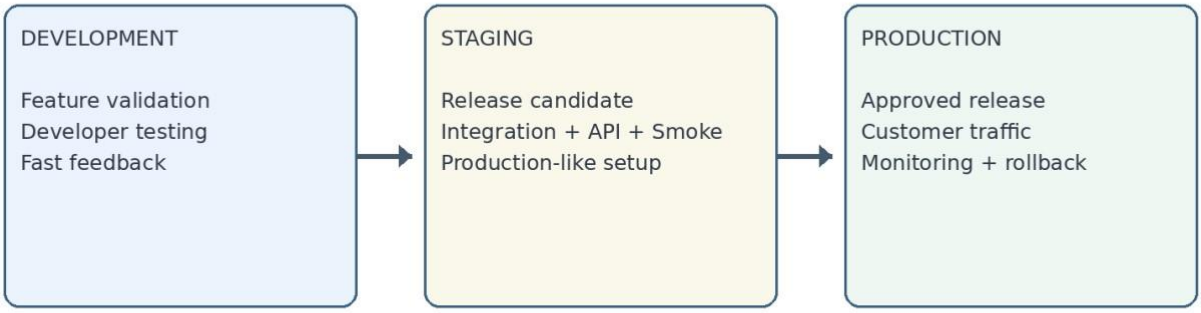


Figure 3 – Automated Testing Pyramid



Pipeline executes fast tests first and progressively deeper validation later.

Figure 4 – Development, Staging and Production Environments



Promotion occurs only after the previous environment passes its quality gates.

Figure 5 – Security and Quality Gates

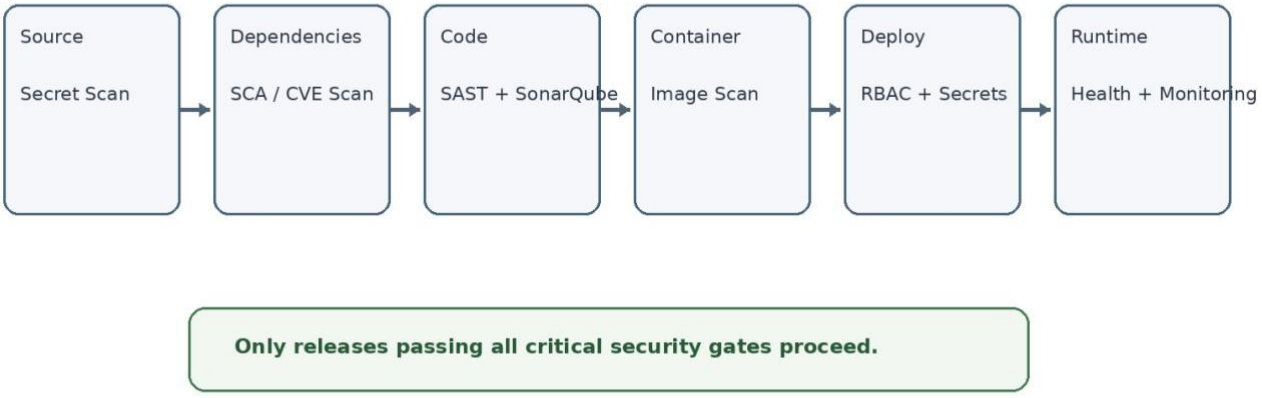
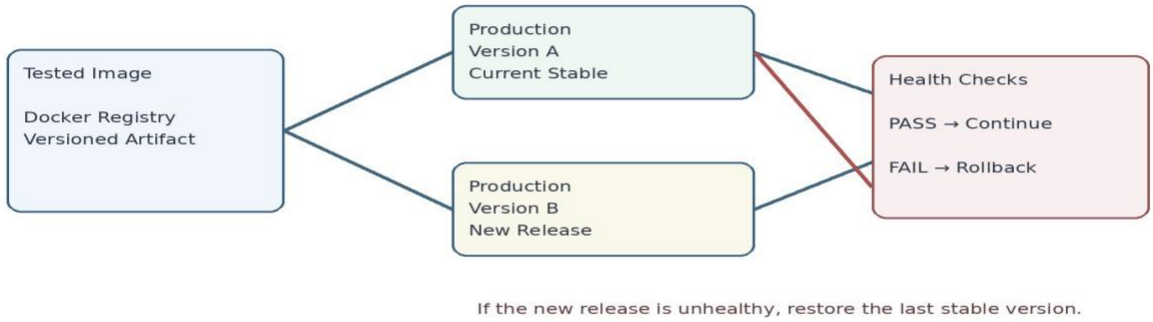
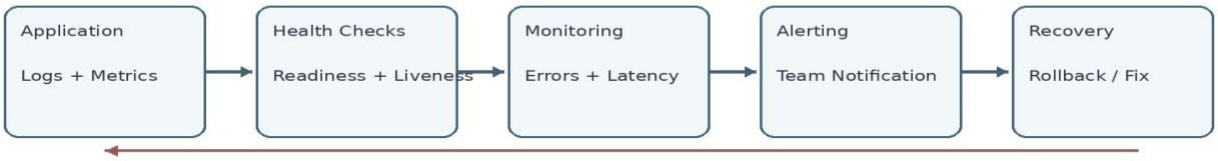


Figure 6 – Production Deployment and Rollback Strategy



If the new release is unhealthy, restore the last stable version.

Figure 7 – Monitoring, Notification and Recovery Loop



Continuous feedback loop improves release reliability.

Conclusion

The proposed CI/CD pipeline provides a structured DevOps workflow for the E-Commerce Order Fulfillment Platform. It connects source management, automated building, testing, code-quality analysis, security checks, container packaging, staging validation, approval, production deployment, monitoring, and rollback. This approach supports frequent and reliable delivery while reducing the risk of defective or insecure releases reaching customers. The design addresses the major assessment areas, including pipeline architecture and workflow, tools, automated testing and quality, deployment, security, rollback, and documents.